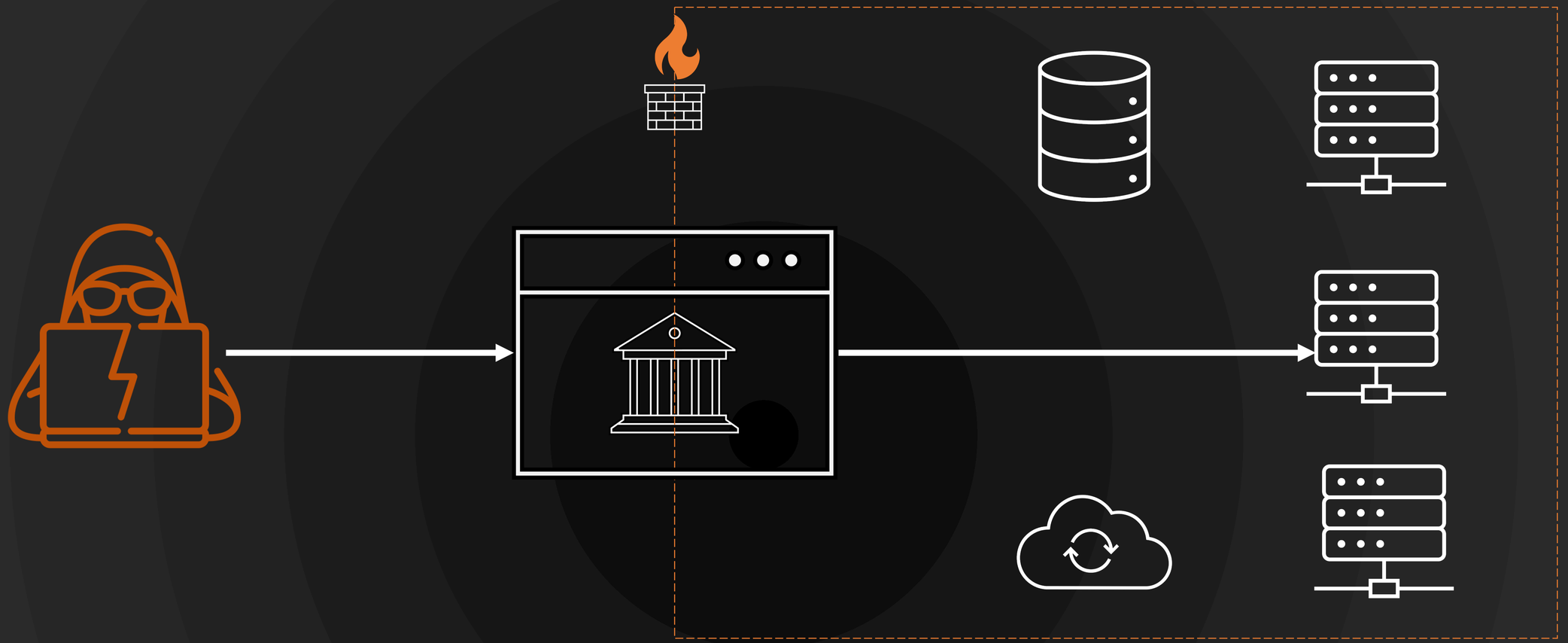


SSRF

Author: Rana Khalil (@rana__khalil)

Agenda

WHAT IS SSRF?

Agenda

WHAT IS SSRF?

HOW DO YOU FIND IT?

Agenda

WHAT IS SSRF?

HOW DO YOU FIND IT?

HOW DO YOU EXPLOIT IT?

Agenda

WHAT IS SSRF?

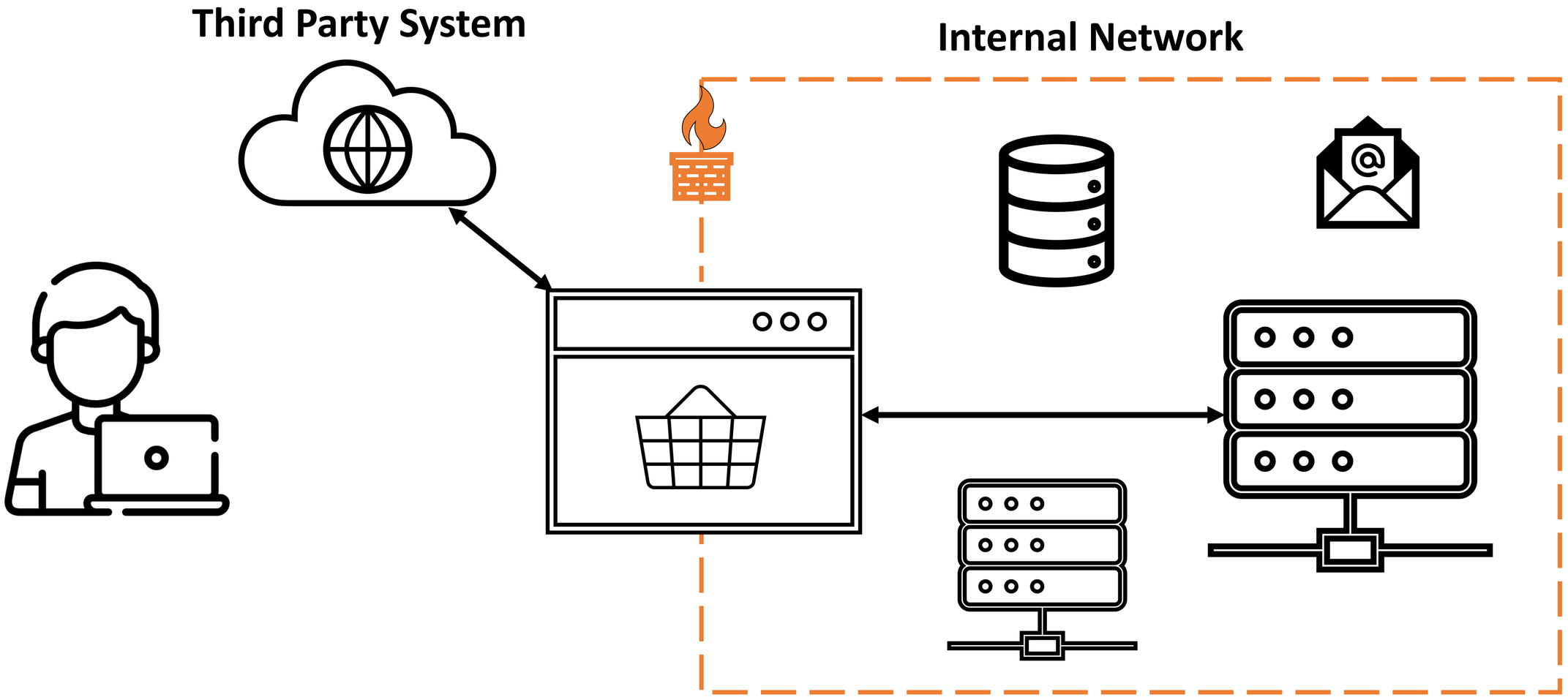
HOW DO YOU FIND IT?

HOW DO YOU EXPLOIT IT?

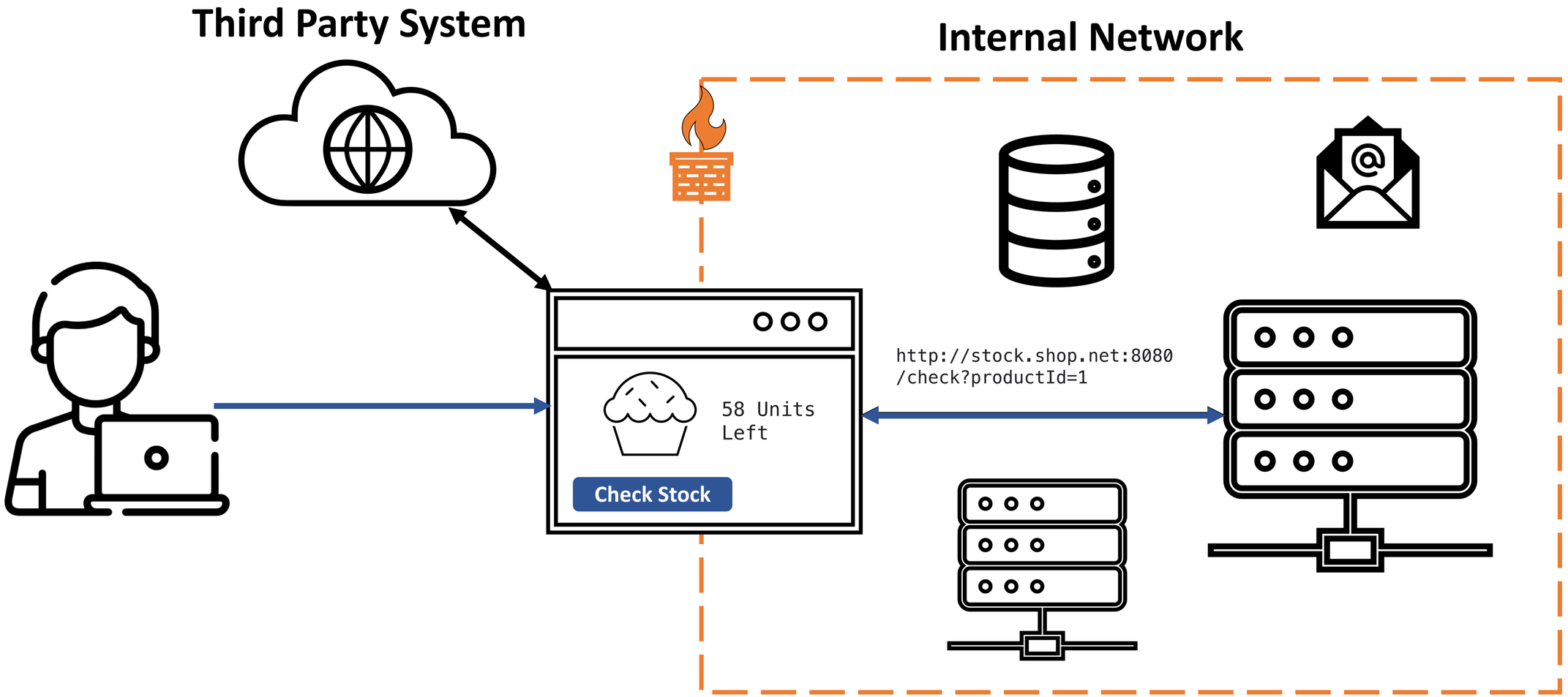
HOW DO YOU PREVENT IT?

WHAT IS SSRF?

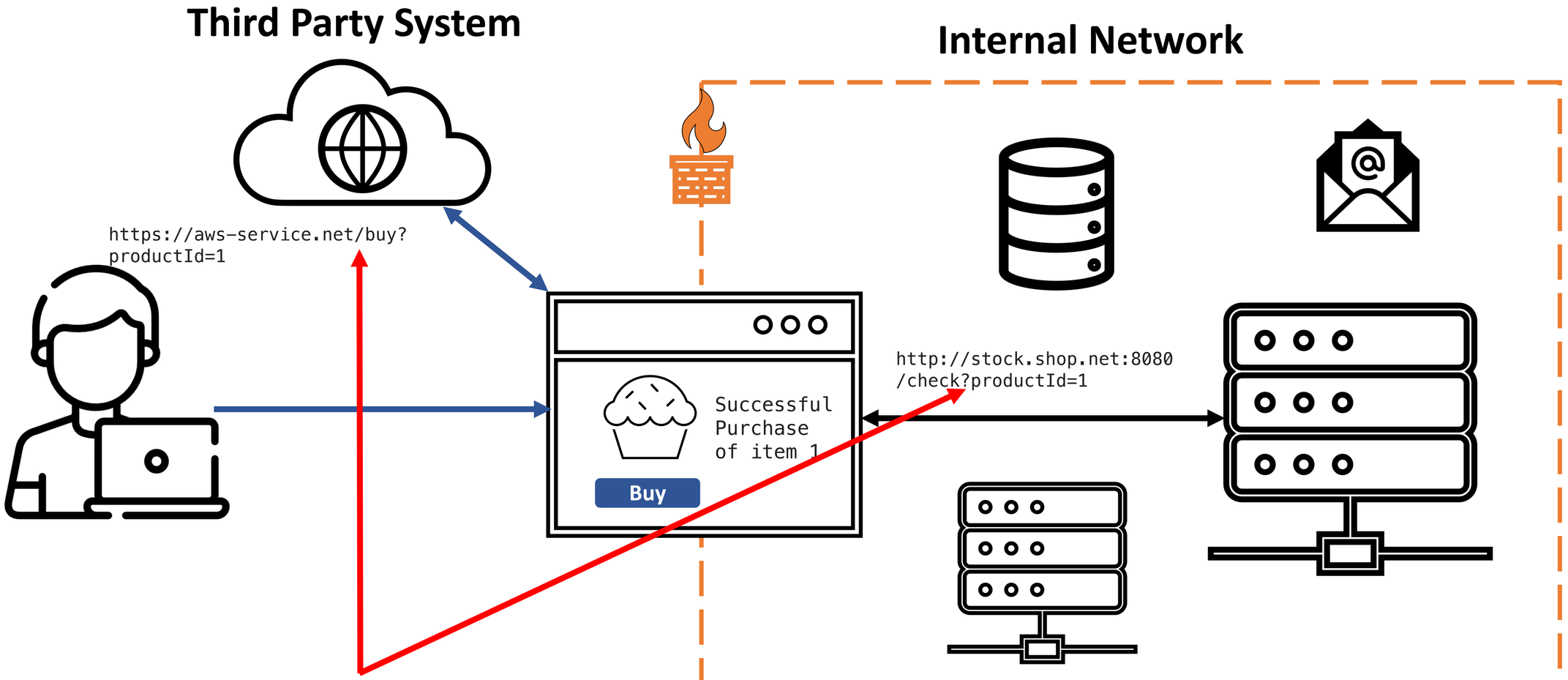
Server-Side Request Forgery (SSRF)



Server-Side Request Forgery (SSRF)

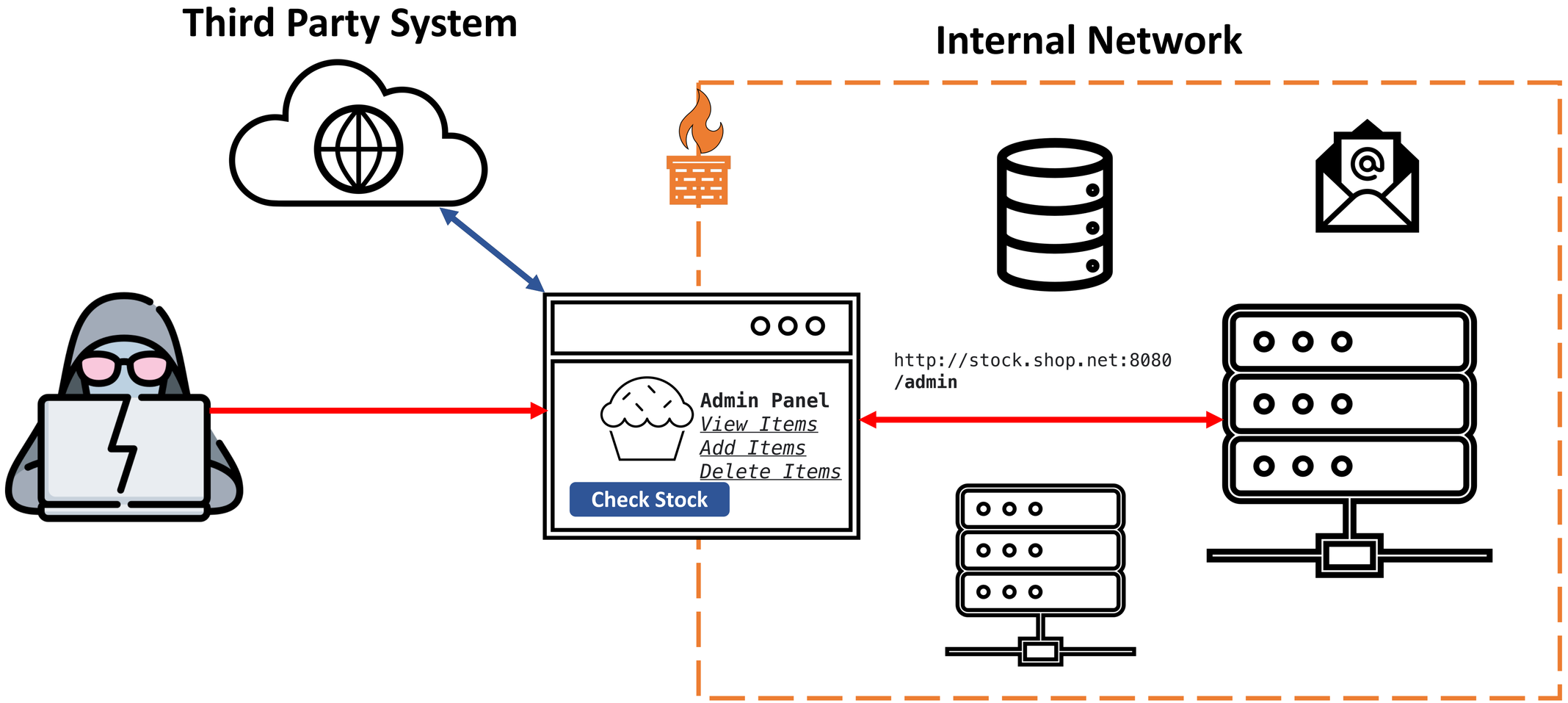


Server-Side Request Forgery (SSRF)

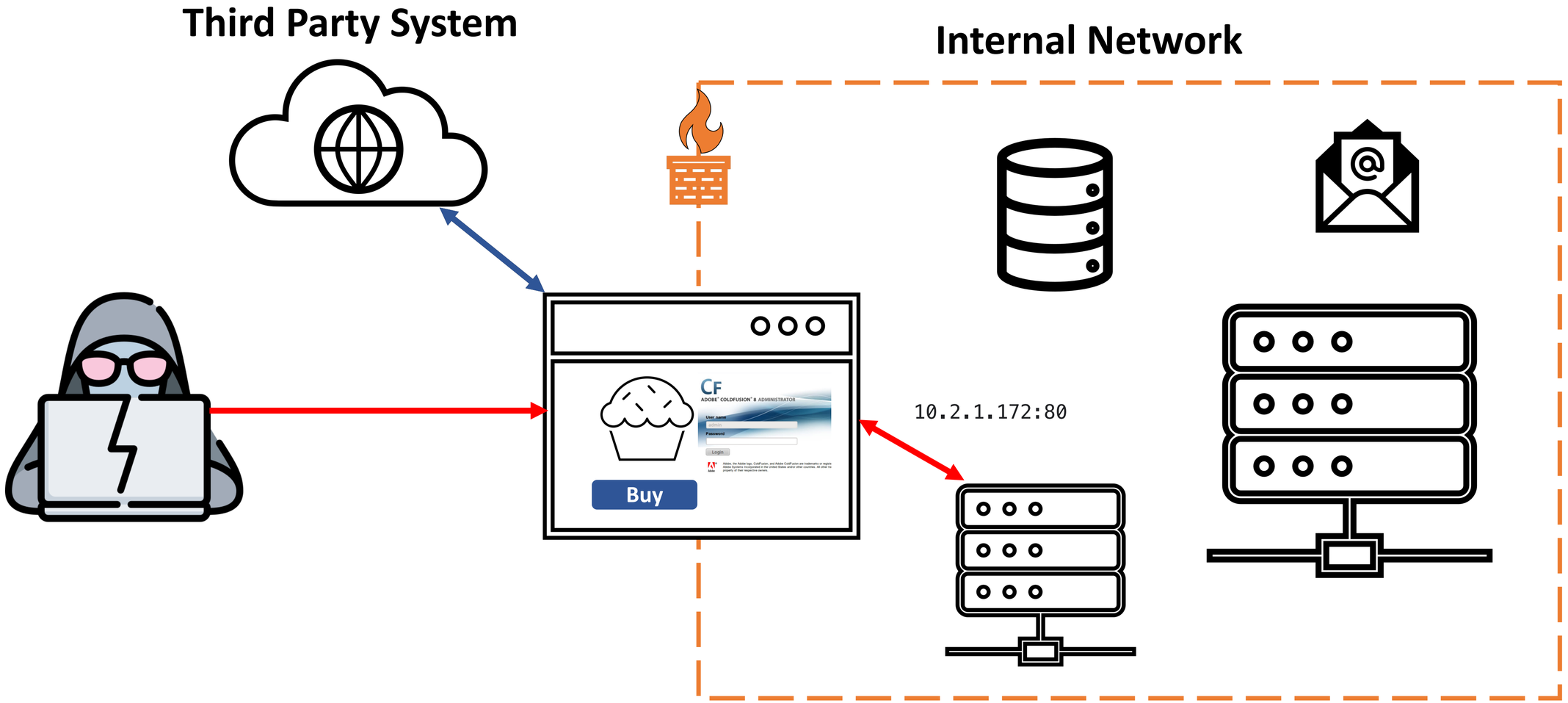


What if the URL parameters are user controllable and not properly validated at the backend?

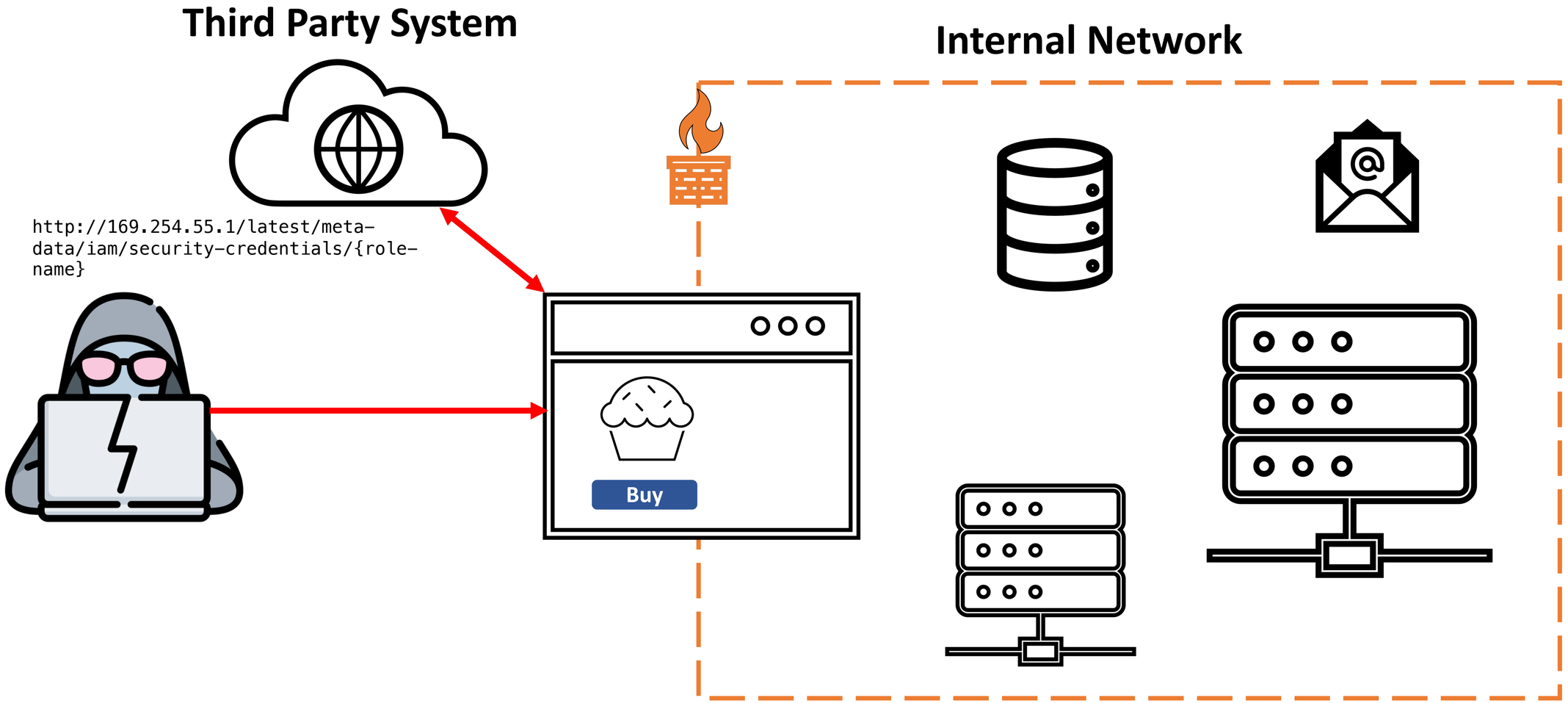
Server-Side Request Forgery (SSRF)



Server-Side Request Forgery (SSRF)



Server-Side Request Forgery (SSRF)



A hacker gained access to 100 million Capital One credit card applications and accounts

By Rob McLean, CNN Business
Updated 5:17 PM EDT, Tue July 30, 2019

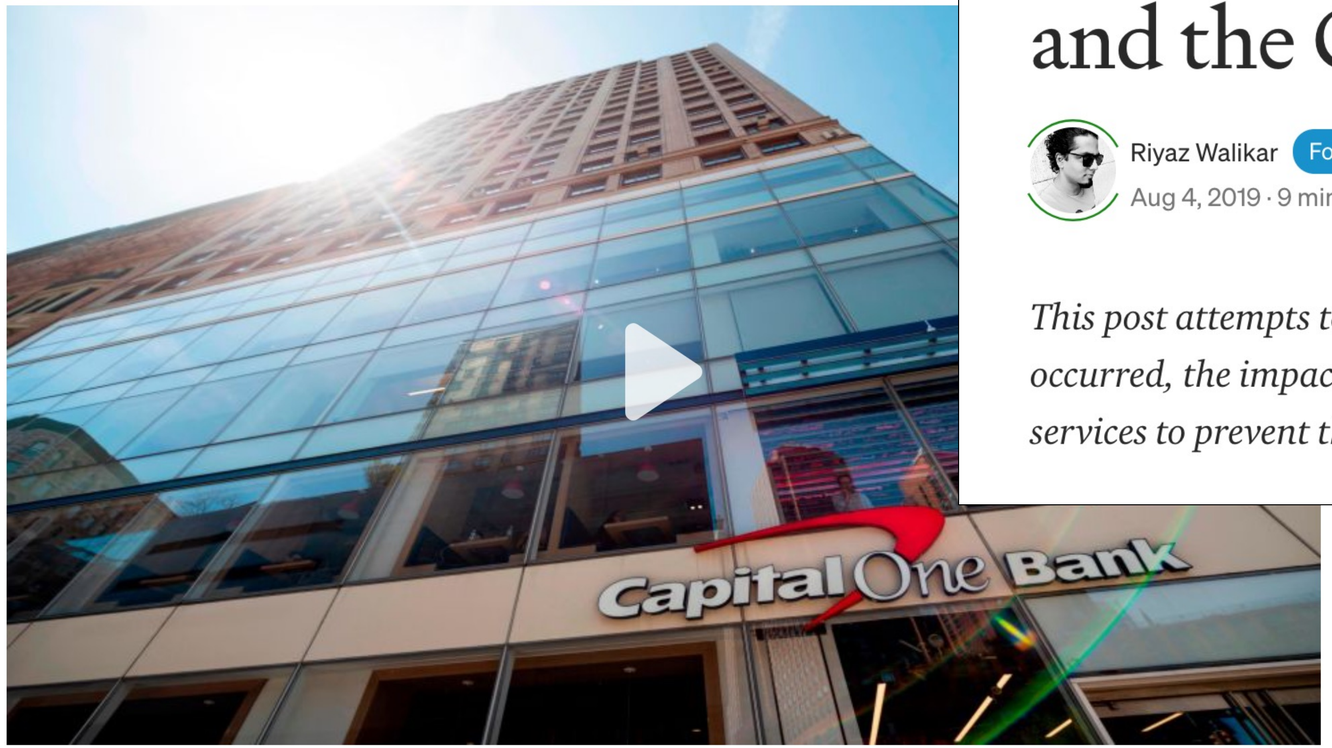


Image Source: https://www.cnn.com/2019/07/29/business/capital-one-data-breach/index.html

An SSRF, privileged AWS keys and the Capital One breach

Riyaz Walikar Follow

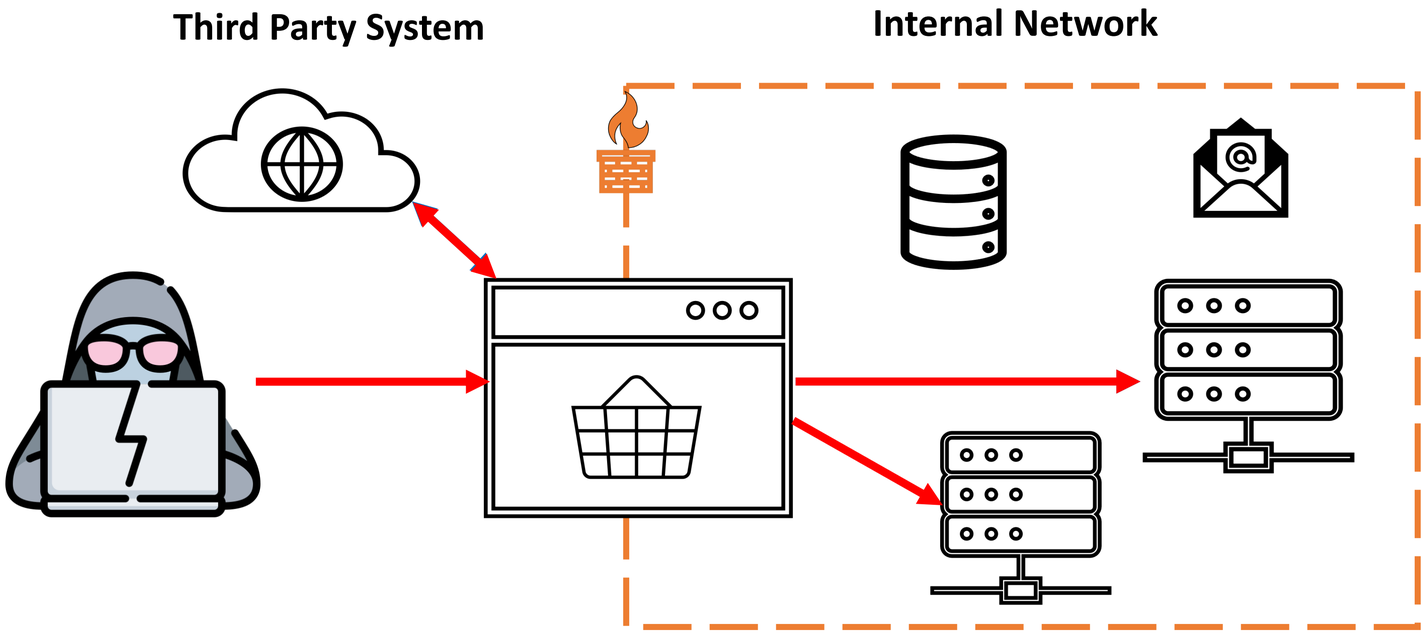
Aug 4, 2019 · 9 min read

This post attempts to explain the technical side of how the Capital One breach occurred, the impact of the breach and what you can do as a user of cloud services to prevent this from happening to you.

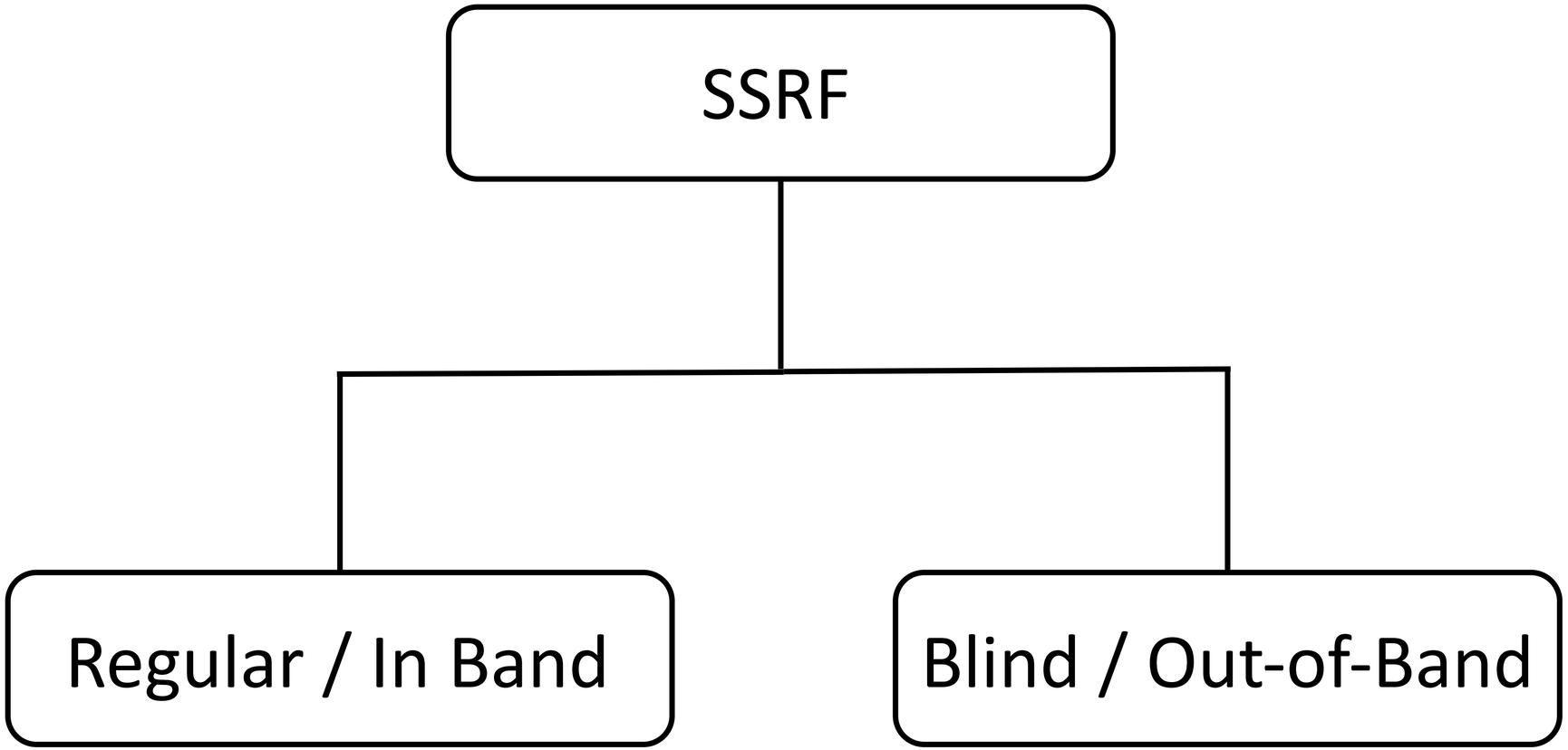
Image Source: https://blog.appsecco.com/an-ssrf-privileged-aws-keys-and-the-capital-one-breach-4c3c2cded3af

Server-Side Request Forgery (SSRF)

SSRF is a vulnerability class that occurs when an application is fetching a remote resource without first validating the user-supplied URL.



Types of SSRF



Impact of SSRF Attacks

- Depends on the functionality in the application that is being exploited
  - **C**onfidentiality – can be None / Partial (Low) / High
  - **I**ntegrity – can be None / Partial (Low) / High
  - **A**vailability – can be None / Partial (Low) / High
- Can lead to sensitive information disclosure, scan of internal network, compromise of internal services, remote code execution, etc.

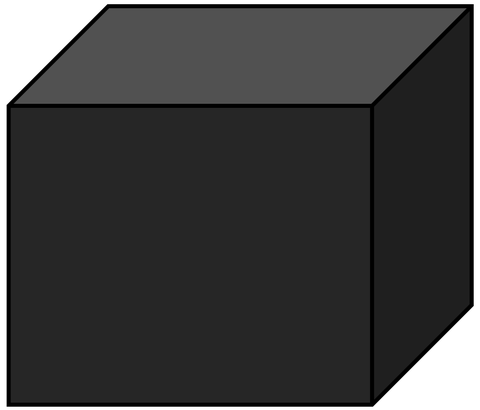
OWASP Top 10

OWASP Top 10 - 2013	OWASP Top 10 - 2017	OWASP Top 10 - 2021
A1 – Injection	A1 – Injection	A1 – Broken Access Control
A2 – Broken Authentication and Session Management	A2 – Broken Authentication	A2 – Cryptographic Failures
A3 – Cross-Site Scripting (XSS)	A3 – Sensitive Data Exposure	A3 - Injection
A4 – Insecure Direct Object References	A4 – XML External Entities (XXE)	A4 – Insecure Design
A5 – Security Misconfiguration	A5 – Broken Access Control	A5 – Security Misconfiguration
A6 – Sensitive Data Exposure	A6 – Security Misconfiguration	A6 – Vulnerable and Outdated Components
A7 – Missing Function Level Access Control	A7 – Cross-Site Scripting (XSS)	A7 – Identification and Authentication Failures
A8 – Cross-Site Request Forgery (CSRF)	A8 – Insecure Deserialization	A8 – Software and Data Integrity Failures
A9 – Using Components with Known Vulnerabilities	A9 – Using Components with Known Vulnerabilities	A9 – Security Logging and Monitoring Failures
A10 – Unvalidated Redirects and Forwards	A10 – Insufficient Logging & Monitoring	A10 – Server-Side Request Forgery (SSRF)

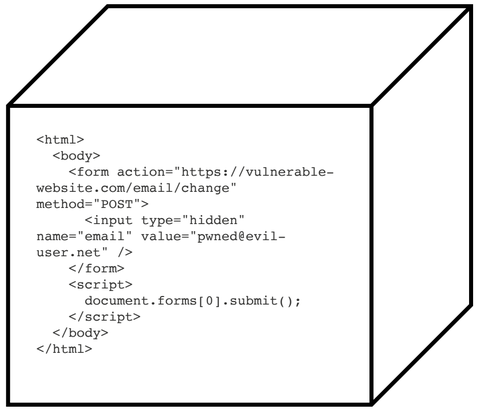
HOW TO FIND SSRF VULNERABILITIES?

Finding SSRF Vulnerabilities

Depends on the perspective of testing.



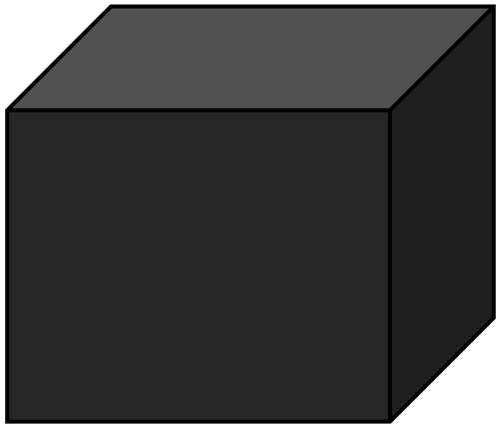
Black Box Testing



White Box Testing

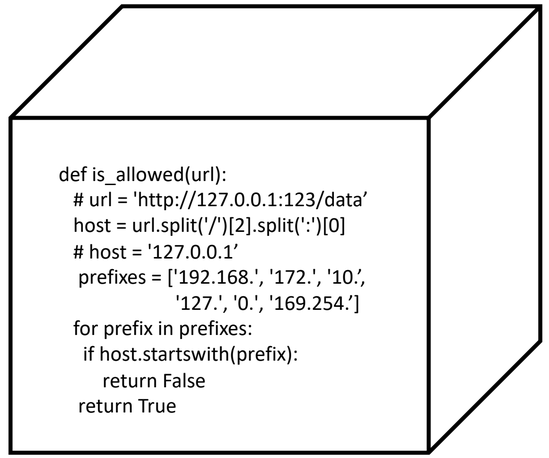
Black-Box Testing Perspective

- Map the application
  - Identify any request parameters that contain hostnames, IP addresses or full URLs
- For each request parameter, modify its value to specify an alternative resource and observe how the application responds
  - If a defense is in place, attempt to circumvent it using know techniques
- For each request parameter, modify its value to a server on the internet that you control and monitor the server for incoming requests
  - If no incoming connections are received, monitor the time taken for the application to respond



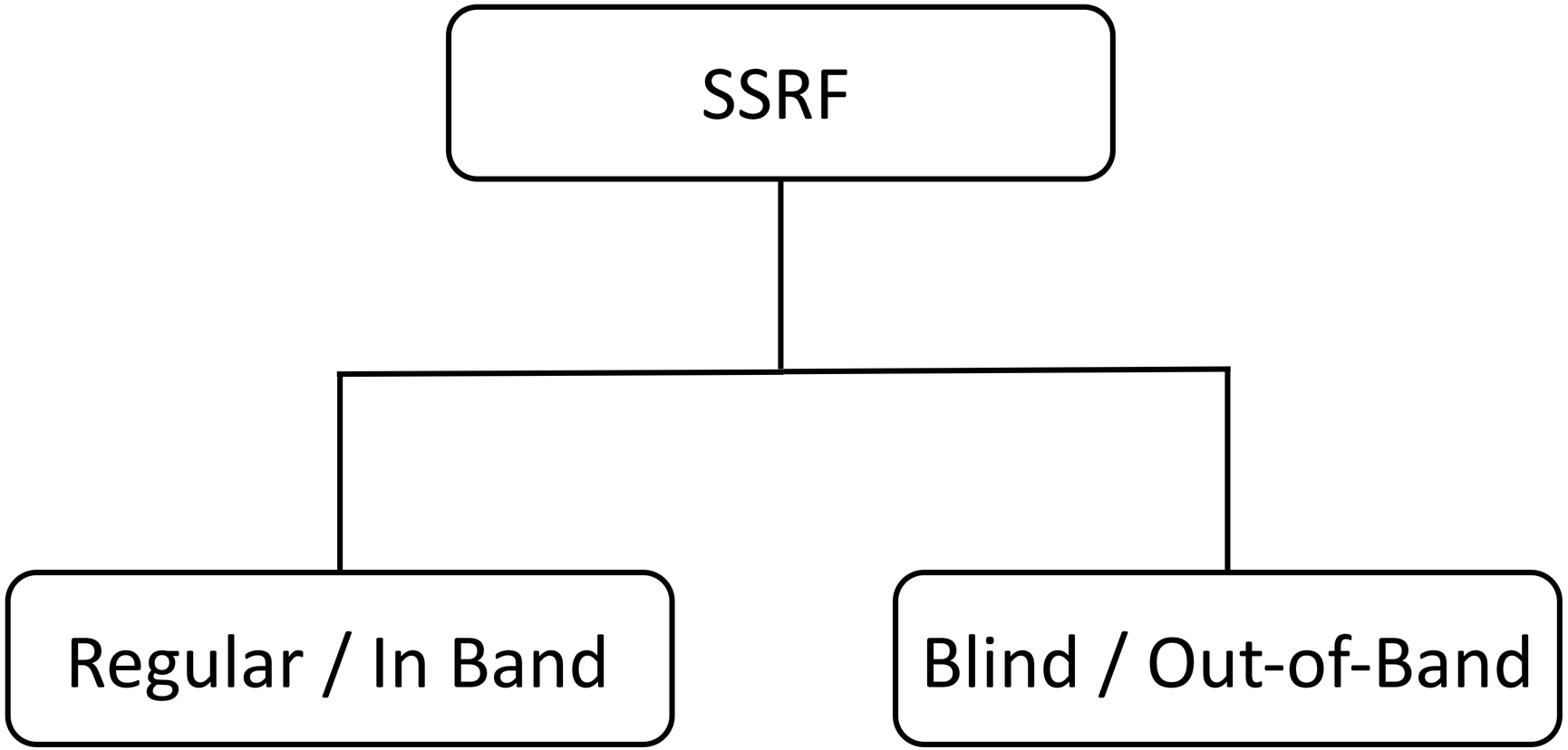
White-Box Testing Perspective

- Review source code and identify all request parameters that accept URLs
  - This could be done by combining both a black-box and white-box testing perspective
- Determine what URL parser is being used and if it can be bypassed. Similarly determine what additional defenses are put in place that can be bypassed
  - A New Era of SSRF - Exploiting URL Parser in Trending Programming Languages by Orange Tsai
- Test any potential SSRF vulnerabilities



HOW TO EXPLOIT SSRF VULNERABILITIES?

Types of SSRF



Exploiting Regular / In-Band SSRF

Request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-
form-urlencoded
Content-Length: 118


stockApi=http://stock.weliketoshop
.net:8080/product/stock/check%3Fpr
oductId%3D6%26storeId%3D1
```

Response:

```
HTTP/1.1 200 OK
Content-Type: text/plain;
charset=utf-8
Connection: close
Content-Length: 3



506
```

Exploiting Regular / In-Band SSRF

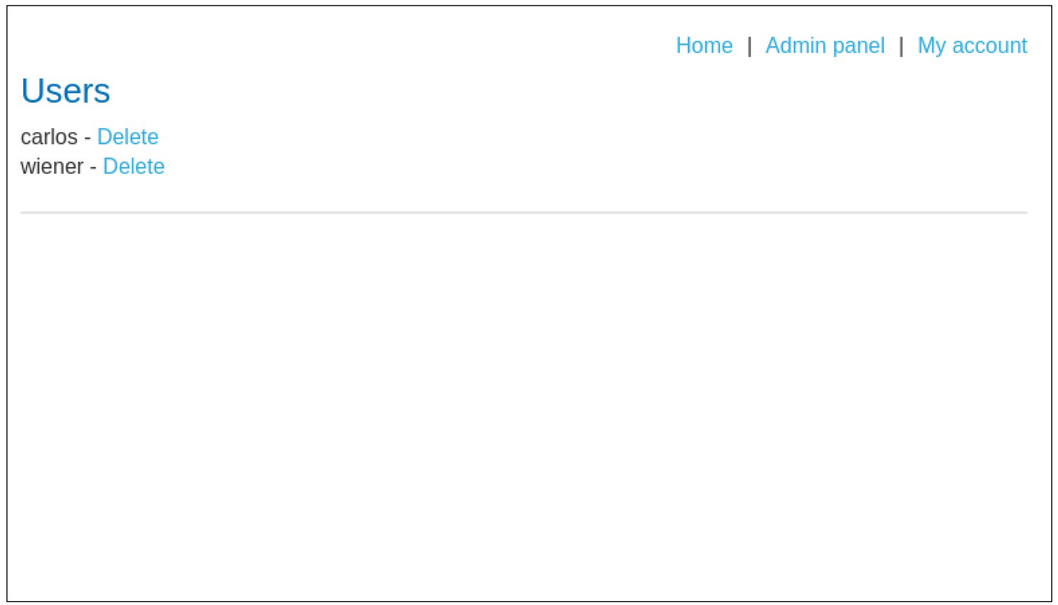
Request:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-
form-urlencoded
Content-Length: 118



stockApi=http://localhost/admin
```

Response:



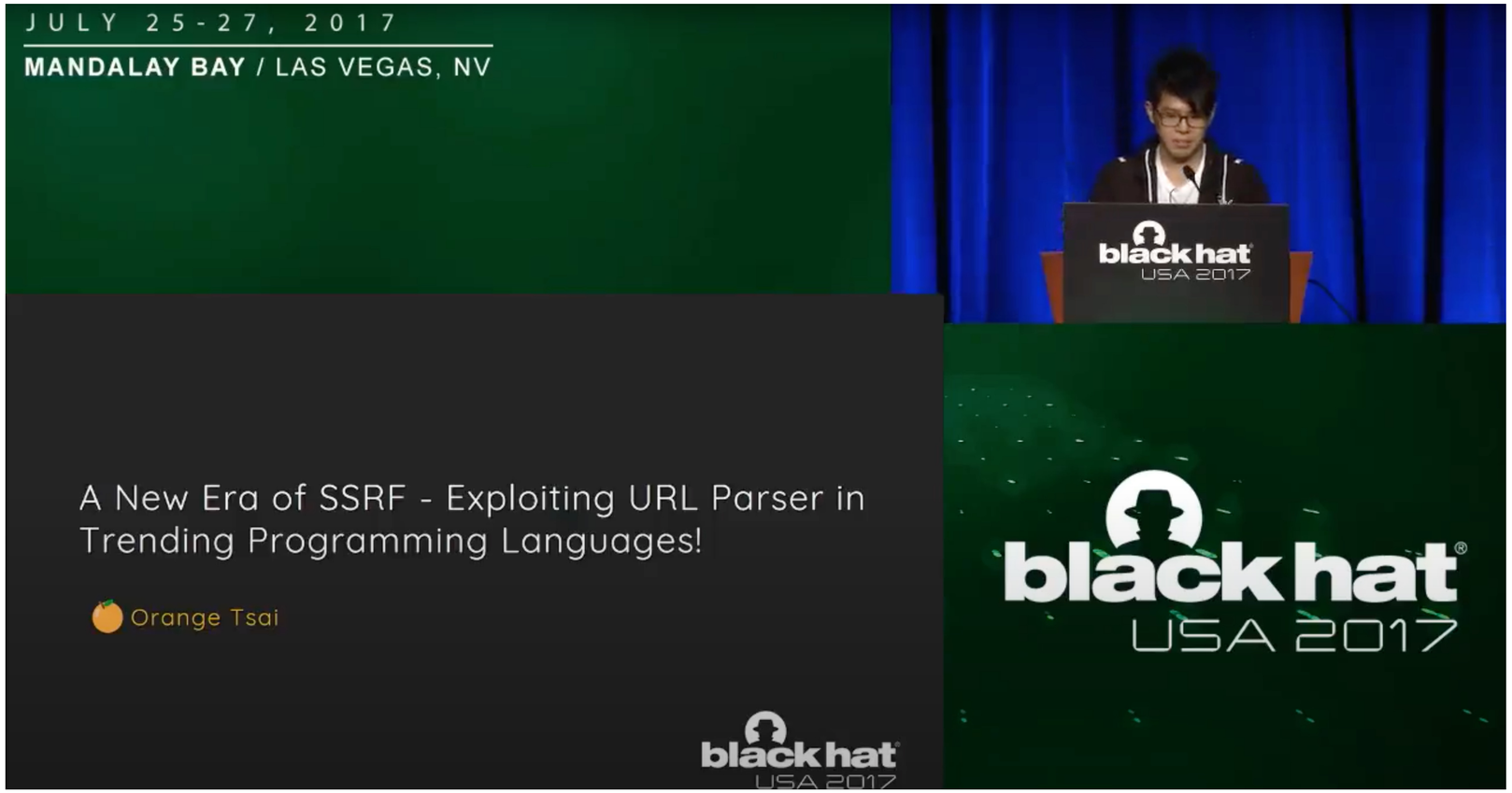
Exploiting Regular / In-Band SSRF

- If the application allows for user-supplied arbitrary URLs, try the following attacks:
  - ❑ Determine if a port number can be specified
  - ❑ If successful, attempt to port-scan the internal network using Burp Intruder
  - ❑ Attempt to connect to other services on the loopback address

Exploiting Regular / In-Band SSRF

- If the application does not allow for arbitrary user-supplied URLs, try to bypass defenses using the following techniques:
  - ❑ Use different encoding schemes
    - decimal-encoded version of 127.0.0.1 is 2130706433
    - 127.1 resolves to 127.0.0.1
    - Octal representation of 127.0.0.1 is 017700000001
  - ❑ Register a domain name that resolves to internal IP address (DNS Rebinding)
  - ❑ Use your own server that redirects to an internal IP address (HTTP Redirection)
  - ❑ Exploit inconsistencies in URL parsing

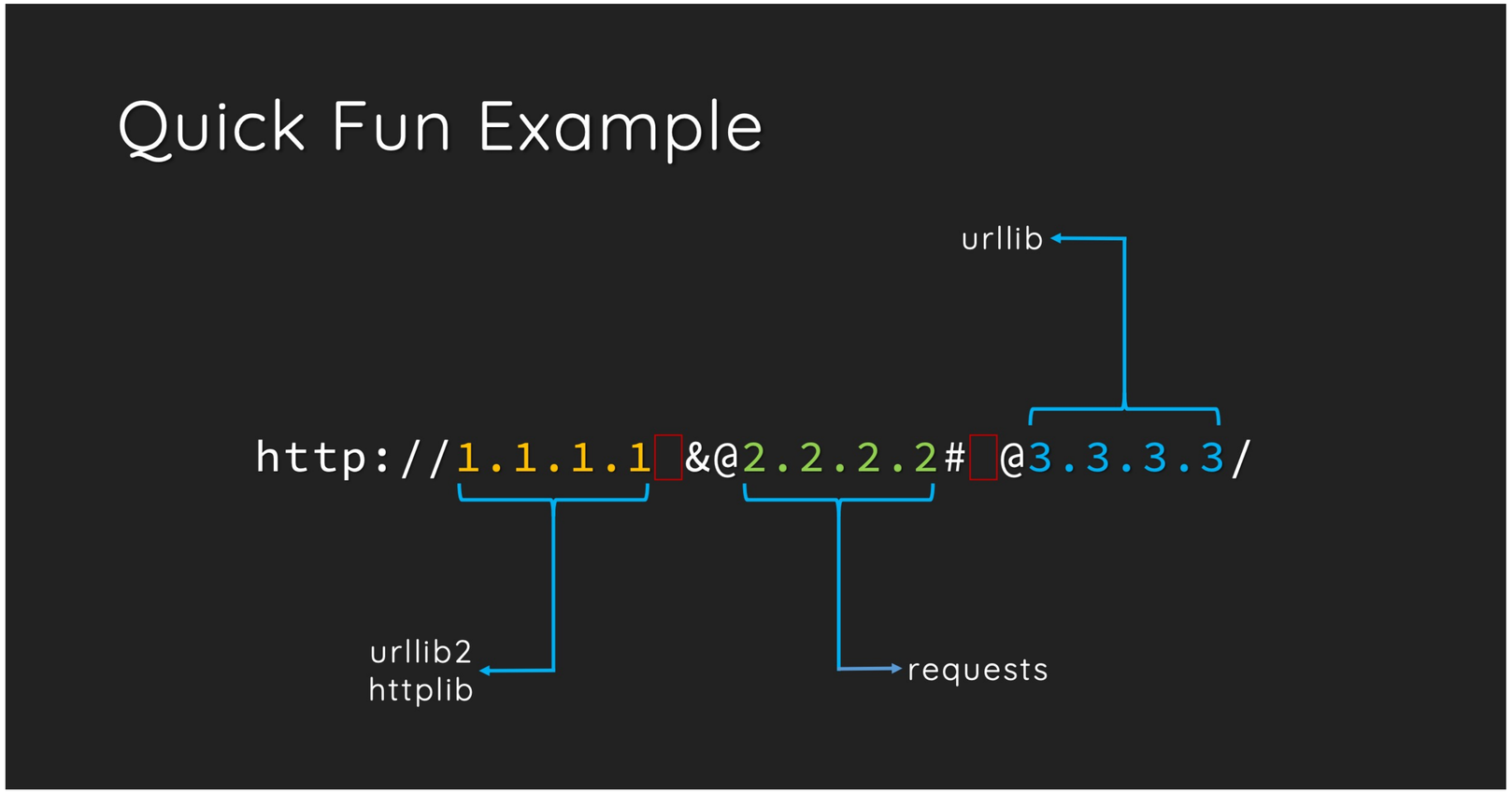
Exploiting Regular / In-Band SSRF



Link to video can be found here.

Link to slides can be found here.

Exploiting Regular / In-Band SSRF



Link to video can be found here.

Link to slides can be found here.

Exploiting Regular / In-Band SSRF

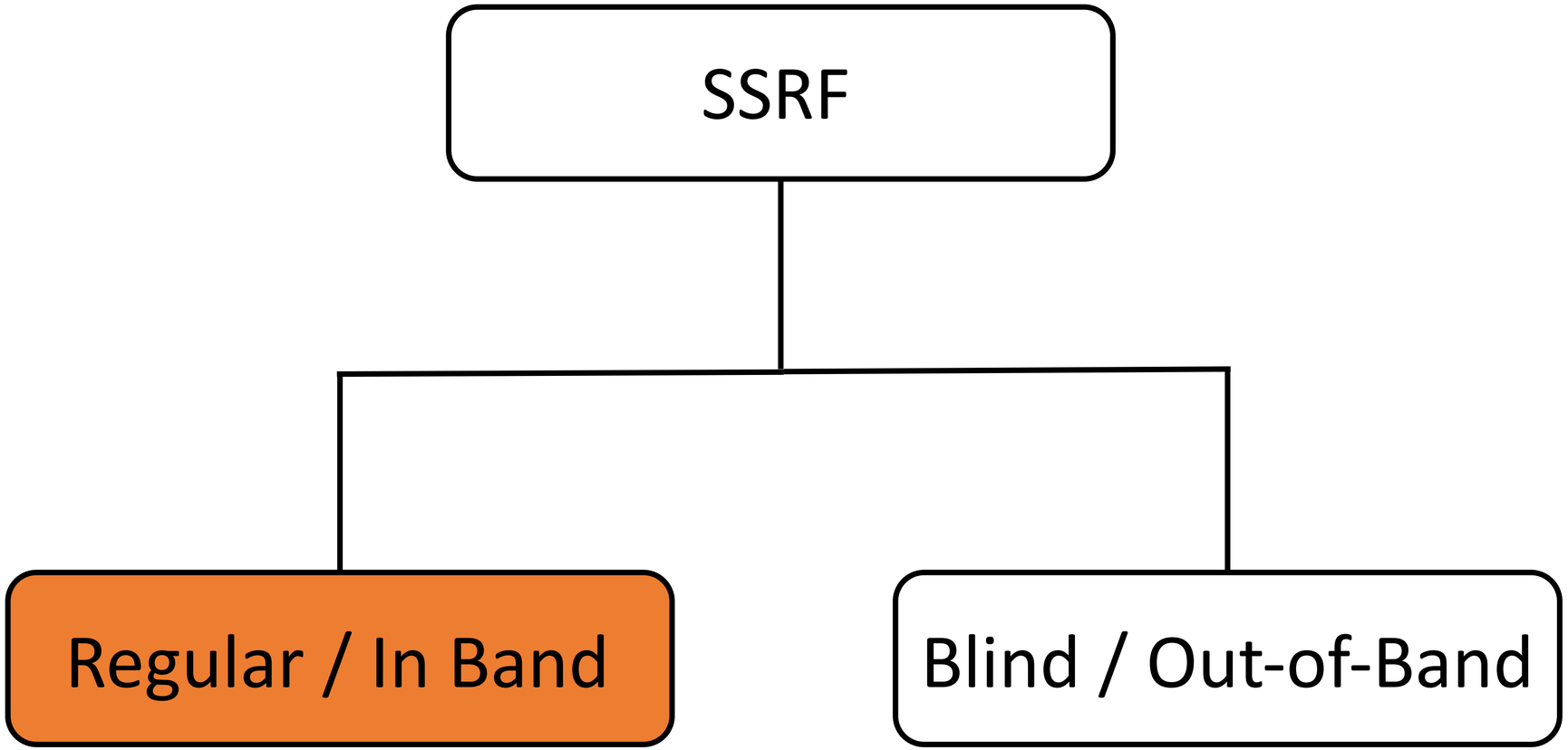
Big Picture

Libraries/Vulns	CR-LF Injection			URL Parsing		
	Path	Host	SNI	Port Injection	Host Injection	Path Injection
Python httplib	💀	💀	💀			
Python urllib		💀	💀		💀	
Python urllib2		💀	💀			
Ruby Net::HTTP	💀	💀	💀			
Java net.URL		💀			💀	
Perl LWP			💀	💀		
NodeJS http	💀					💀
PHP http_wrapper				💀	💀	
Wget		💀	💀			
cURL				💀	💀	

Link to video can be found here.

Link to slides can be found here.

Types of SSRF

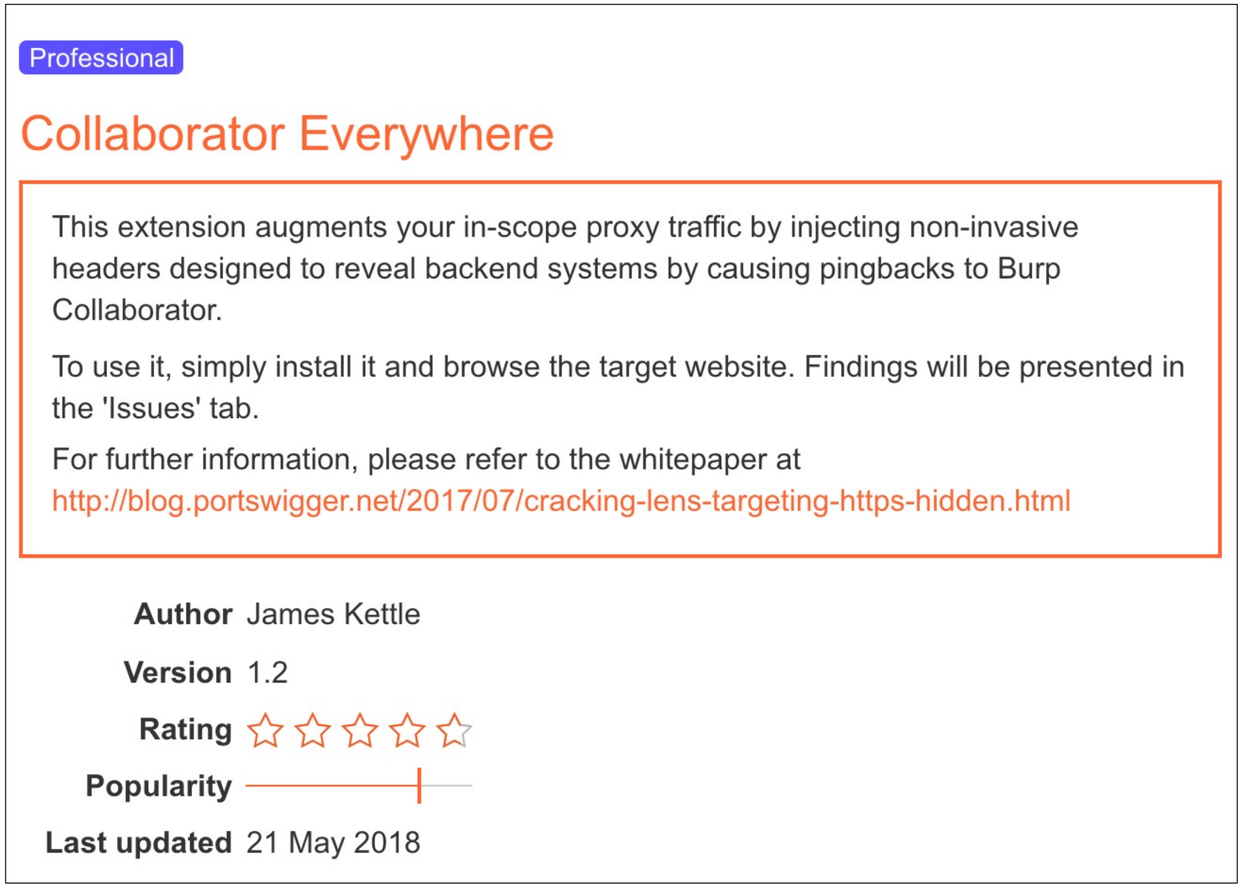


Exploiting Blind / Out-of-Band SSRF

- If the application is vulnerable to blind SSRF, try to exploit the vulnerability using the following techniques:
  - ❑ Attempt to trigger an HTTP request to an external system you control and monitor the system for network interactions from the vulnerable server
    - Can be done using Burp Collaborator
  - ❑ If defenses are put in place, use the techniques mentioned in the previous slides to obfuscate the external malicious domain
  - ❑ Once you've proved that the application is vulnerable to blind SSRF, you need to determine what your end goal is
    - An example would be to probe for other vulnerabilities on the server itself or other backend systems

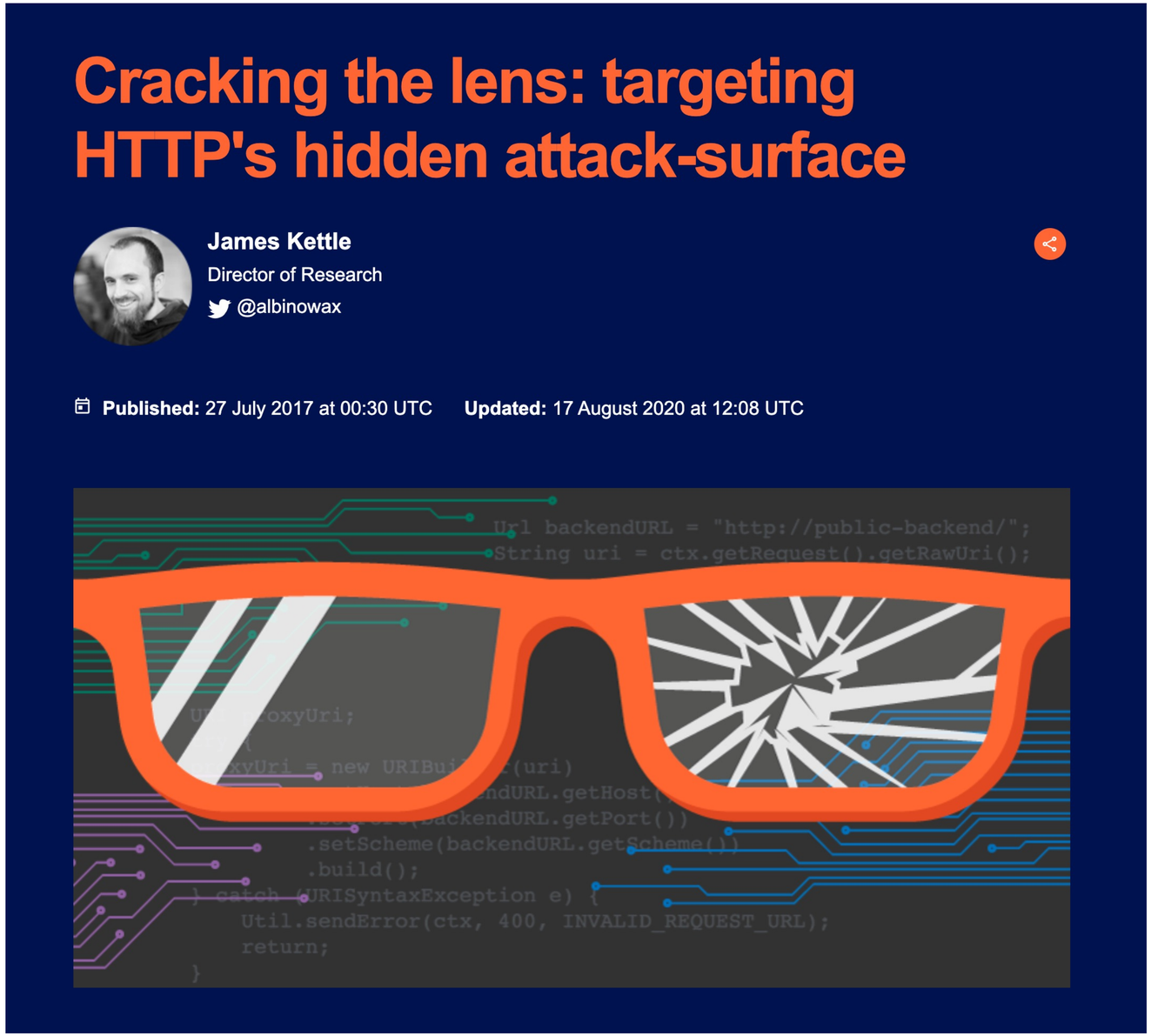
Collaborator Everywhere

Burp Suite ***Collaborator Everywhere*** Extension



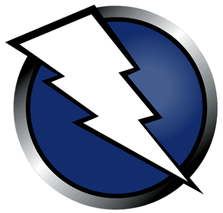
Cracking the Lens: Targeting HTTP's Hidden Attack-surface

Article Link:
https://portswigger.net/research/cracking-the-lens-targeting-https-hidden-attack-surface



Automated Exploitation Tools

Web Application Vulnerability Scanners (WAVS).

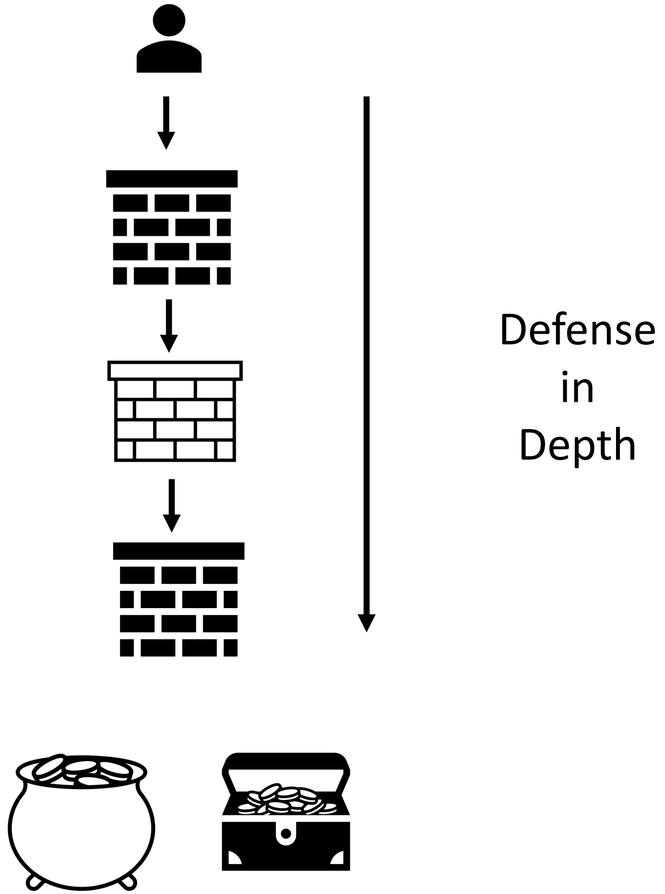


HOW TO PREVENT SSRF VULNERABILITIES?

Preventing SSRF Vulnerabilities

Defense in depth approach:

- Application Layer defenses
- Network Layer Defenses



Application Layer Defenses

- Sanitize and validate all client-supplied input data
- Enforce the URL schema, port, and destination with a positive allow list
- Do not send raw responses to clients
- Disable HTTP redirections

Note: Do not mitigate SSRF vulnerabilities using deny lists or regular expressions.

Network Layer Defenses

- Segment remote resource access functionality in separate networks to reduce the impact of SSRF
- Enforce “deny by default” firewall policies or network access control rules to block all but essential intranet traffic

Resources

- Web Security Academy - SSRF
  - https://portswigger.net/web-security/ssrf
- Web Application Hacker's Handbook
  - *Chapter 10 – Attacking Back-End Components (pgs. 390 – 392)*
- OWASP – SSRF
  - https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
- Server-Side Request Forgery Prevention Cheat Sheet
  - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
- SSRF Bible Cheat Sheet
  - https://cheatsheetseries.owasp.org/assets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet_SSRF_Bible.pdf
- Preventing Server-Side Request Forgery Attacks
  - https://seclab.nu/static/publications/sac21-prevent-ssrf.pdf
- A New Era of SSRF - Exploiting URL Parser in Trending Programming Languages!
  - https://www.blackhat.com/docs/us-17/thursday/us-17-Tsai-A-New-Era-Of-SSRF-Exploiting-URL-Parser-In-Trending-Programming-Languages.pdf